Автономная некоммерческая организация профессионального образования
«Международная Академия Информационных Технологий «ИТ ХАБ» (АНО ПО «ИТ ХАБ»)
115093, г. Москва, вн. тер. г. муниципальный округ Даниловский, ул. Дубининская, д. 96
ИНН 9709063913 КПП 772501001
+7 (499) 350-66-04
Ithub.ru

Специальность

10.02.05

(шифр)

дневное

(отделение)

КОНТРОЛЬНАЯ ТОЧКА № 5

Автоматизированное модульное тестирование с использованием базы данных

(тема практической работы)

Выполнил студент группы 4ИБ1-.23

К.А. Мызочкин

Москва, 2026

Цель и задачи работы

Цель работы: реализовать автоматизированную проверку алгоритма определения атак на шахматного короля с использованием модульных тестов JUnit 5 и тестовых данных из базы SQLite. Дополнительно организовать сохранение истории запусков и оценить покрытие кода средствами JaCoCo.

Задачи: подготовить набор тестовых случаев; подключить SQLite к Java-проекту; выполнить параметризованные проверки; убедиться, что тестирование выявляет намеренно внесенную ошибку; сохранить результаты в базе данных; сформировать отчеты JUnit и JaCoCo.

1. Используемые средства и структура проекта

Работа выполнена в GitHub Codespaces. Для сборки и запуска применялся Gradle, для модульного тестирования JUnit 5, для хранения входных данных и результатов SQLite, для анализа покрытия JaCoCo.

Исходный проект размещен в приватном репозитории: https://github.com/bancik222/KT5.

Основные файлы проекта: build.gradle, settings.gradle, src/main/java/ru/ithub/chess/ChessMath.java, src/test/java/ru/ithub/chess/ChessDatabaseTest.java, src/test/java/ru/ithub/chess/TestResultImporter.java, database/chess_tests.db и scripts/flip_case.py.

2. Организация тестирования

Тестовые случаи хранятся в таблице test_cases базы SQLite. Тест ChessDatabaseTest получает их через JDBC и передает в параметризованный тест JUnit 5 с использованием @MethodSource. Для каждого набора координат проверяются признаки атаки ладьей и слоном, общий результат, а также обработка некорректных входных данных.

Подготовлено семь сценариев: одновременная атака двух фигур, блокировка ладьи слоном, блокировка слона ладьей, атака только слоном, отсутствие атаки, совпадение координат фигур и выход координат за границы доски.

```
gradle clean test
```

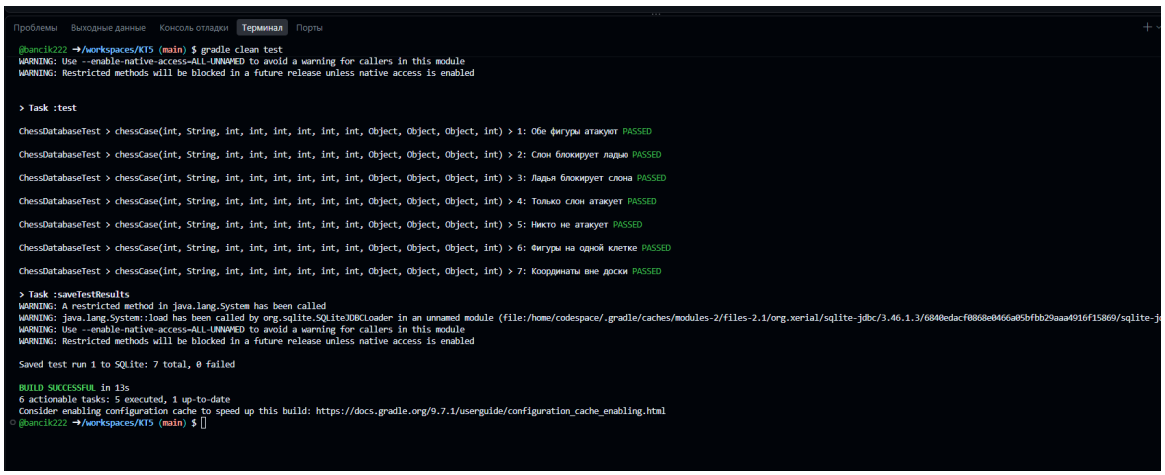


Рисунок 1. Семь тестов завершились успешно, результаты записаны в SQLite.

3. Проверка обнаружения ошибки

Для демонстрации работы механизма контроля ожидаемый результат первого тестового случая был намеренно изменен с помощью скрипта flip_case.py. При повторном запуске один из семи тестов завершился с ошибкой, что привело к состоянию BUILD FAILED. При этом сведения о запуске были сохранены в SQLite.

python3 scripts/flip_case.py

gradle test --rerun-tasks

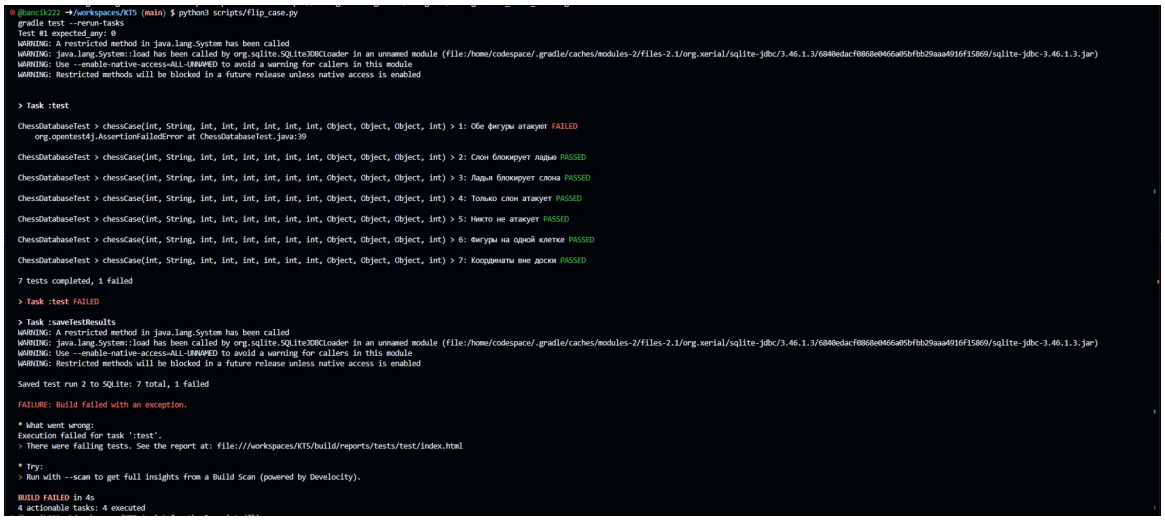


Рисунок 2. JUnit обнаружил намеренно созданное расхождение: 1 ошибка из 7 тестов.

После повторного запуска flip_case.py ожидаемый результат был восстановлен. Повторное тестирование снова завершилось успешно.

4. Сохранение истории запусков

В базе SQLite ведется история запусков. Для каждого запуска сохраняются дата и время, число тестов, число успешных и неуспешных проверок, продолжительность выполнения. По выводу SQL-запроса подтверждены четыре записи.

Запуск	Всего	Успешно	Ошибок

1	7	7	0
2	7	6	1
3	7	7	0
4	7	7	0

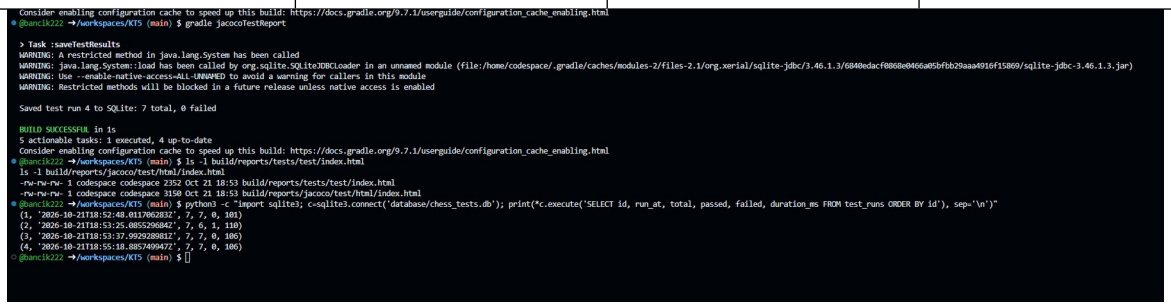


Рисунок 3. Наличие HTML-отчетов и история четырех запусков в SQLite.

5. Анализ покрытия кода

После выполнения тестов сформирован отчет JaCoCo. Фактические значения покрытия получены из файла jacocoTestReport.xml.

Метрика	Покрытие
Инструкции (Instruction)	64,4%
Ветвления (Branch)	67,9%
Строки (Line)	55,6%

Покрытие показывает долю кода, задействованную при тестировании. Значения ниже 100% означают, что часть путей выполнения и строк программы не была охвачена выбранными сценариями. Это не противоречит успешному прохождению всех семи тестов.



Рисунок 4. Фактические показатели покрытия JaCoCo из XML-отчета.

6. Вывод

В ходе контрольной точки № 5 организовано автоматизированное модульное тестирование шахматного алгоритма с загрузкой тестовых данных из SQLite. Проведены успешные запуски, а также эксперимент с заведомо неверным ожидаемым результатом. JUnit выявил ошибку, Gradle отразил неуспешный запуск, а история проверки была сохранена в базе. После восстановления данных все семь тестов вновь прошли успешно.

Отчеты JUnit и JaCoCo сформированы. Покрытие инструкций составило 64,4%, ветвлений 67,9%, строк 55,6%. Для повышения покрытия в дальнейшем можно добавить проверки оставшихся граничных ситуаций и ветвей алгоритма.